

=====

GUÍA DE INSTALACIÓN — GesmedWeb en un servidor nuevo (Ubuntu 26.04)

=====

Última actualización: 2026-09-04

Esta guía instala GesmedWeb desde cero en un servidor limpio, usando:

- Dump de ESTRUCTURA (no create_all() de SQLModel) para regenerar el esquema de la base de datos, ya que mis_modelos.py todavía tiene diferencias cosméticas de tipo/nullable respecto a la BD real (ver migrations/verificar_estructura.py).
- migrations/parametros_fijos.py para poblar los catálogos fijos.
- migrations/crear_admin_inicial.py para crear el primer usuario ADMIN.

NOTA: la base de datos "geodata" (creada originalmente para separar el perfil de usuarios) quedó en desuso al pasar a cifrado AES-256-GCM sobre la tabla points de gesmed. No se crea ni se usa en esta guía. El código muerto que la referenciaba (GesmedWeb/querys/points.py y la variable _URL_GEODATA en querys.py) fue eliminado del proyecto.

1. SISTEMA BASE

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3 python3-venv python3-pip git \
  mariadb-server mariadb-client build-essential \
  nodejs npm unzip curl
```

2. MARIADB

```
sudo mysql_secure_installation
```

```
sudo mysql -e "
CREATE DATABASE gesmed CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
CREATE USER 'med_admin'@'localhost' IDENTIFIED BY '<password-fuerte>';
GRANT ALL PRIVILEGES ON gesmed.* TO 'med_admin'@'localhost';
FLUSH PRIVILEGES;
"
```

Solo se necesita la base "gesmed". No se crea "geodata" (ver nota arriba).

3. ESTRUCTURA DE LA BASE DE DATOS (dump de estructura, no create_all())

Desde tu gesmed local (o el servidor de referencia), generar un dump SOLO de estructura (sin datos de pacientes reales):

```
mysqldump --no-data --routines --triggers \
  -h localhost -u med_admin -p gesmed > sql/gesmed_estructura.sql
```

Importante: sql/gesmed.sql (el dump completo actual del repo) NO sirve para esto tal cual — contiene datos reales de pacientes (paciente, atencion, diagnostico, etc.), no es un artefacto de instalación limpio. Generar gesmed_estructura.sql como se indica arriba y usar ESE archivo como el artefacto oficial de instalación (guardarlo en el repo junto a parametros_fijos.sql).

Copiar el archivo al servidor nuevo y aplicarlo:

```
mysql -u med_admin -p gesmed < sql/gesmed_estructura.sql
```

4. CÓDIGO + ENTORNO PYTHON

```
git clone <tu-repo> /opt/gesmedweb # o rsync si no se usa git
cd /opt/gesmedweb
python3 -m venv .virtual
.virtual/bin/pip install -r requirements.txt
```

5. VARIABLES DE ENTORNO (.env)

Generar la clave maestra de cifrado:

```
python3 -c "import secrets; print(secrets.token_hex(32))"
```

Crear el archivo .env en la raíz del proyecto:

```
GESMED_MASTER_KEY=<el hex generado arriba>
GESMED_DB_URL=mysql+pymysql://med_admin:<password>@localhost:3306/gesmed
```

(GEODATA_DB_URL ya no es necesaria — se puede omitir del .env.)

CRÍTICO: si este servidor va a recibir datos migrados desde amaymed (paciente, diagnostico, prescripcion — todos con campos cifrados AES-256-GCM), GESMED_MASTER_KEY debe ser EXACTAMENTE la misma que se usó para cifrar esos datos originalmente. Si es una instalación 100% nueva sin datos heredados, puede generarse una clave nueva como arriba.

6. CATÁLOGOS Y USUARIO ADMIN INICIAL

```
.virtual/bin/python -m migrations.parametros_fijos --instalar
```

```
.virtual/bin/python -m migrations.crear_admin_inicial \
--usuario admin --nombre "Administrador" --email admin@dominio.com
```

(pedirá la contraseña por prompt oculto, nunca como argumento en texto plano)

7. VERIFICACIÓN DE ESTRUCTURA (recomendado)

```
.virtual/bin/python -m migrations.verificar_estructura
```

Confirma que el dump de estructura aplicado coincide con lo que mis_modelos.py espera, antes de arrancar la app en producción.

8. MIGRACIÓN DE DATOS DESDE AMAYMED (solo si se restituye desde amaymed)

Este paso NO aplica a una instalación 100% nueva sin datos heredados —

sáltalo si no hay una base amaymed de origen. Aplica cuando el servidor nuevo debe quedar con los datos clínicos migrados desde amaymed (paciente, diagnóstico, atención, prescripción, exámenes, etc.).

8.1 Prerrequisito: GESMED_MASTER_KEY debe ser la misma que se usó para cifrar los datos migrados originalmente (ver sección 5, punto CRÍTICO) — si es distinta, los campos AES-256-GCM quedan ilegibles/corruptos tras la migración.

8.2 Restituir (o dejar accesible) la base amaymed de origen en el servidor nuevo, y agregar sus credenciales al .env:

```
AMAYMED_DB_HOST=localhost
AMAYMED_DB_PORT=3306
AMAYMED_DB_USER=med_admin
AMAYMED_DB_PASSWORD=<password>
AMAYMED_DB_NAME=amaymed
```

(Estas variables tienen fallback a los valores por defecto vistos en migrations/orquestador.py, pero es mejor declararlas explícitamente.)

8.3 Ejecutar el orquestador de migración, uno por cada médico a migrar:

```
.virtual/bin/python -m migrations.orquestador --medico-id <id>
```

Esto hace, en un solo proceso (ver migrations/orquestador.py):

1. Backup automático de amaymed y gesmed antes de tocar nada
2. Aplica los ALTER TABLE necesarios (columnas a BLOB para cifrado)
3. Trunca las tablas destino en gesmed (orden inverso de FK)
4. Migra tabla por tabla en orden de dependencias FK:
paciente → diagnostico → atencion → rel_atencion_diagnostico →
prescripcion → prescripcion_cuidados → certificado →
exámenes_grupo → exámenes_catalogo → exámenes_pedido →
pedido_imagen
5. Verifica conteos origen vs destino por tabla
6. Reporte final; si algo falla, no continúa a ciegas

8.4 Upsert de tablas no cubiertas por el orquestador (si aplica):

```
.virtual/bin/python -m migrations.upsert_medicamento_presentacion
```

(gesmed.presentacion_medicamentos → gesmed.medicamento_presentacion, upsert por id_presentacion — ver el docstring del script antes de correrlo, sobrescribe destino por id)

8.5 Revisar el reporte final del orquestador (conteos origen/destino por tabla) y los errores impresos por consola antes de dar la migración por buena. Repetir 8.3 por cada médico si hay más de uno.

9. LEVANTAR LA APP

```
reflex init    # primera vez: descarga bun, instala deps del frontend
reflex run --env prod
```

Para producción real (proceso persistente + reverse proxy), lo habitual

es agregar sobre esto:

- reflex export (build de frontend + backend para producción)
- un servicio systemd para mantener el proceso corriendo
- nginx como reverse proxy (TLS, dominio, etc.)

Esto no está incluido en esta guía — pedirlo aparte si se necesita.

10. ACCESO REMOTO VÍA VPN (WireGuard)

Objetivo: nadie accede a GesmedWeb ni a MariaDB directamente por internet. Solo se expone el puerto de WireGuard (UDP); la app y la BD quedan escuchando únicamente en la IP privada de la VPN.

10.1 Instalar WireGuard en el servidor

```
sudo apt install -y wireguard
```

10.2 Generar claves del servidor

```
wg genkey | sudo tee /etc/wireguard/server_private.key
sudo chmod 600 /etc/wireguard/server_private.key
sudo cat /etc/wireguard/server_private.key | wg pubkey | \
  sudo tee /etc/wireguard/server_public.key
```

10.3 Configurar la interfaz del servidor

```
sudo nano /etc/wireguard/wg0.conf
```

```
[Interface]
Address = 10.10.0.1/24
ListenPort = 51820
PrivateKey = <contenido de server_private.key>
# Habilita forwarding/NAT solo si el cliente necesita salir a
# internet a través del servidor; para acceso solo a GesmedWeb no
# hace falta.
```

```
[Peer]
# Un bloque [Peer] por cada equipo cliente que se conecte
PublicKey = <clave pública del cliente>
AllowedIPs = 10.10.0.2/32
```

10.4 Levantar la interfaz y habilitarla al boot

```
sudo systemctl enable --now wg-quick@wg0
sudo wg show
```

10.5 Abrir el puerto en el firewall (ufw)

```
sudo ufw allow 51820/udp
sudo ufw allow OpenSSH
sudo ufw enable
```

IMPORTANTE: no abrir los puertos de Reflex (3000/8000) ni de MariaDB (3306) al exterior — solo deben ser alcanzables dentro de la VPN (ver 9.7).

10.6 Configurar el cliente (equipo remoto)

En el equipo desde el que se accederá remotamente:

```
sudo apt install -y wireguard # o WireGuard app en Windows/macOS
wg genkey | tee client_private.key | wg pubkey > client_public.key
```

```
sudo nano /etc/wireguard/wg0.conf
```

```
[Interface]
```

```
Address = 10.10.0.2/24
```

```
PrivateKey = <contenido de client_private.key>
```

```
[Peer]
```

```
PublicKey = <server_public.key del servidor>
```

```
Endpoint = <IP-pública-del-servidor>:51820
```

```
AllowedIPs = 10.10.0.1/32 # solo tráfico hacia el servidor gesmed
```

```
PersistentKeepalive = 25
```

```
sudo wg-quick up wg0
```

Copiar la clave pública del cliente (client_public.key) al bloque [Peer] del servidor (paso 9.3) antes de conectar.

10.7 Restringir MariaDB a la VPN (no exponerla a la red pública)

```
sudo nano /etc/mysql/mariadb.conf.d/50-server.cnf
```

```
bind-address = 10.10.0.1 # solo la IP de la VPN, no 0.0.0.0
```

```
sudo systemctl restart mariadb
```

Y el GESMED_DB_URL en .env sigue apuntando a localhost/127.0.0.1 en el propio servidor (la app y la BD corren en la misma máquina) — el bind-address solo importa si algún día separas la BD a otro host dentro de la VPN.

10.8 Probar la conexión

Desde el equipo cliente, con la VPN levantada:

```
ping 10.10.0.1
```

```
ssh usuario@10.10.0.1 # si SSH también se restringe a la VPN
```

```
curl http://10.10.0.1:3000 # si reflex run expone ahí el frontend
```

Si reflex run escucha en 0.0.0.0 por defecto, ya es alcanzable desde 10.10.0.2 sin abrir ningún puerto adicional al público — el único punto de entrada expuesto a internet es el puerto UDP de WireGuard.

10.9 Checklist rápido para mañana

- [] wireguard instalado en servidor y equipo cliente
- [] claves generadas en ambos lados
- [] wg0.conf servidor con [Peer] del cliente
- [] wg0.conf cliente con Endpoint = IP pública del servidor
- [] ufw: solo 51820/udp y SSH abiertos al público
- [] mariadb bind-address restringido a la IP de la VPN
- [] ping + curl exitoso entre cliente y servidor por la VPN

```
=====
```

RESUMEN DE ORDEN DE EJECUCIÓN

```
=====
```

1. apt install (sistema + mariadb + node)
 2. crear BD gesmed + usuario med_admin
 3. mysql gesmed < gesmed_estructura.sql
 4. git clone + venv + pip install -r requirements.txt
 5. crear .env (GESMED_MASTER_KEY, GESMED_DB_URL)
 6. python -m migrations.parametros_fijos --instalar
 7. python -m migrations.crear_admin_inicial --usuario ...
 8. python -m migrations.verificar_estructura (opcional, para confirmar)
 9. [solo si se restituye desde amaymed] python -m migrations.orchestrador
--medico-id <id> (repetir por médico) + upsert_medicamento_presentacion
 10. reflex init && reflex run --env prod
 11. WireGuard: instalar, generar claves, configurar wg0.conf en ambos
lados, restringir ufw y bind-address de MariaDB a la VPN
- ```
=====
```